

LangGraph Fundamentals - Course Reference

Chapter 1: State, Nodes and Edges

Every LangGraph application is built from three primitives. State is a shared dictionary that every step can read and write. Nodes are plain Python functions that receive the state and return only the keys they changed. Edges determine which node runs next. The graph is compiled before it can be invoked.

Chapter 2: Checkpointers and Memory

A checkpointer saves the full graph state after every node, keyed by a thread_id

This single addition gives an application conversation memory, time travel to earlier checkpoints, and recovery after a crash. InMemorySaver keeps state in RAM and is lost on restart. SqliteSaver writes to a file and survives restarts.

Chapter 3: Dynamic Fan-out with Send

The Send primitive enables dynamic fan-out. A router function returns a list of Send objects, one per work item, and LangGraph schedules all of them in parallel

The number of branches is decided at runtime rather than at compile time. A reducer such as operator.add is required to merge the parallel writes.

Chapter 4: Human in the Loop

The interrupt function pauses a graph mid-node and saves a checkpoint. The caller receives the interrupt payload and can later resume with Command(resume=value). A checkpointer is mandatory for interrupts to work. Any code with side effects should be placed after the interrupt, because the node re-runs from its start on resume.